

Ultra Fast Encrypted Network Traffic Classification Based on Classification By Retrieval Powered By Out-of-Distribution

Yehonatan Zion^{a,*}, Amit Dvir^a, Chen Hajaj^b, Ran Dubin^a, Neriya Filber^c

^a*Department of Computer and Software Engineering, Ariel Cyber Innovation Center,
Ariel University, Israel*

^b*Department of Industrial Engineering and Management, Data Science and Artificial
Intelligence Research Center, Ariel University, Israel*

^c*Department of Computer Science, Ariel Cyber Innovation Center, Ariel
University, Israel*

Abstract

The dominance of encryption and the relentless growth of network traffic have rendered traditional deep packet inspection ineffective and made retraining-based machine learning classifiers operationally costly, as every new application or threat class requires a full model rebuild. In order to overcome this challenge, we present an ultra-fast encrypted traffic classification system based on Classification By Retrieval (CBR): each flow is represented by a fixed-length feature vector extracted from only the first few packets, indexed in a HNSW approximate nearest-neighbor structure, and classified by retrieving its nearest labeled neighbor. New classes are absorbed by inserting a handful of labeled vectors into the running index, with no retraining and no downtime, and out-of-distribution traffic is detected within the same re-

*Corresponding author

Email addresses: yehonata.zion@msmail.ariel.ac.il (Yehonatan Zion),
amitdv@ariel.ac.il (Amit Dvir), chenha@ariel.ac.il (Chen Hajaj),
rand@ariel.ac.il (Ran Dubin), neriya.filber@msmail.ariel.ac.il (Neriya Filber)

trieval step using a single global distance threshold. We evaluate handcrafted statistical features and learned CNN and MLP embeddings against a Random Forest baseline on the public CESNET-TLS22 dataset (187 application classes) and a proprietary cellular dataset (Radcom). With CNN embeddings, CBR matches Random Forest accuracy (macro F1 of 0.923 versus 0.925 on CESNET) while training roughly ten times faster, adapts to new classes two to three orders of magnitude faster (seconds versus thousands of seconds), and detects out-of-distribution flows with up to 0.90 AUROC. These results show that retrieval-based classification is a practical and adaptable alternative for real-time encrypted traffic analysis in dynamic network environments.

Keywords: Encrypted Network Traffic Classification, Classification By Retrieval, Approximate Nearest Neighbors, out-of-distribution Detection, Few-Shot Learning, Network Security

List of Abbreviations

ANN	Approximate Nearest Neighbor
AUROC	Area Under the ROC Curve
CBR	Classification By Retrieval
CNN	Convolutional Neural Network
DoH	DNS over HTTPS
DPI	Deep Packet Inspection
HC	Handcrafted
HNSW	Hierarchical Navigable Small World
ML	Machine Learning
MLP	Multi-Layer Perceptron
NN	Nearest Neighbor
OOD	Out-of-Distribution
RF	Random Forest
SNI	Server Name Indication
SOTA	State-of-the-Art
TLS	Transport Layer Security

1. Introduction

The analysis of internet traffic has grown increasingly complex due to two primary factors: the widespread adoption of encryption protocols and the exponential increase in traffic volume. Encryption now dominates the majority of internet traffic, largely driven by the adoption of standards such as Transport Layer Security (TLS). This trend reflects a broader industry movement toward secure connections spurred by growing privacy and se-

curity awareness. Initiatives like Let's Encrypt [1] have further supported this shift, enabling more websites and services to adopt encryption seamlessly. While encryption enhances privacy and security for legitimate users, it also introduces significant challenges for traditional Deep Packet Inspection (DPI) traffic analysis methods. As a result, understanding and monitoring encrypted traffic have become critical focus areas for network security and traffic analysis efforts.

In recent years, significant changes to internet protocols, such as QUIC [2], HTTP/3, TLS 1.3, and DNS over HTTPS (DoH) [3], have further complicated traffic classification. Traditional DPI methods, which rely on identifiers like DNS queries and Server Name Indication (SNI) to analyze encrypted traffic, are quickly becoming obsolete. Machine Learning (ML) models have emerged as a potential solution for encrypted traffic classification. However, these models require extensive labeled datasets for training and can only classify data into pre-defined categories. When new applications or traffic types emerge, these models become outdated. Updating them involves collecting data for the new category, integrating it into the existing dataset, and retraining the model, a resource-intensive process that poses significant operational challenges.

The dramatic increase in Internet traffic volume presents another significant challenge. A recent Ericsson study [4] reported that global cellular network data traffic reached 145 exabytes (EB) per month at the end of Q1 2024, marking a growth 6% quarter-over-quarter from Q4 2023. Video content alone accounted for 73% of all cellular data traffic at the close of 2023. This relentless growth, projected to continue at an average annual

rate of 20% until 2029, underscores the need for faster, more accurate traffic analysis methods. Classifying this amount of data in real time is a new challenge.

To address these challenges, our approach performs Classification By Retrieval (CBR): it leverages approximate nearest-neighbor (ANN) search to efficiently classify new vectors by matching them to a prelabeled, indexed collection. This enables real-time classification and allows the system to incorporate new traffic classes without requiring retraining. If the distance of a sample to all known classes exceeds a predefined threshold, it is classified as out-of-distribution (OOD), thereby facilitating the detection of emerging or unknown traffic patterns. The system can dynamically expand with minimal labeled data from new classes, ensuring adaptability. Furthermore, by relying solely on statistical features, it remains robust against potential protocol changes.

1.1. Contributions

In summary, the contributions of this paper are as follows:

- **A retraining-free retrieval framework for encrypted traffic classification.** We represent each flow by a fixed-length vector extracted from only the first few packets, and classify it by ANN retrieval over a labeled HNSW index. New samples and entirely new classes are absorbed by inserting their vectors into the running index at $\mathcal{O}(\log n)$ cost, with no model retraining and no downtime.
- **Unified, training-free out-of-distribution detection.** OOD flows are flagged within the same retrieval step using a single global distance

threshold that is distribution-free and requires no per-class statistics, unifying classification and OOD detection in one operation.

- **Substantial efficiency gains in training and adaptation.** On a state-of-the-art (SOTA) public dataset (CESNET-TLS22), CBR matches the Random Forest (RF) baseline in accuracy (CNN-CBR macro F1 of 0.923 versus 0.925) while training roughly ten times faster, and it adapts to new classes two to three orders of magnitude faster (seconds versus thousands of seconds).
- **A comprehensive comparison across the two ends of the feature spectrum.** We evaluate handcrafted statistical features, and learned CNN and MLP embeddings, each against an RF baseline, across packet budgets, few-shot regimes, and OOD detection, on a SOTA public dataset (CESNET-TLS22, 187 classes) and a real-world industry dataset (the proprietary cellular Radcom dataset).

2. Related work

Machine Learning and Deep Learning have become the dominant approaches for encrypted network traffic classification [5, 6, 7, 8, 9, 10, 11]. Despite strong classification performance, these works share a fundamental limitation: they treat the class set as fixed, and require full model retraining whenever new traffic classes appear — a costly and operationally impractical assumption in dynamic network environments. Retrieval-based classification bypasses this retraining requirement by classifying new queries against a pre-indexed reference set, making feature representation a central design

decision.

We therefore survey the two dominant feature paradigms that the literature has explored for retrieval-based traffic classification: handcrafted statistical vectors and end-to-end learned embeddings. Biernacki [12] demonstrates that statistical throughput features such as average bit rate, variance, and extrema are highly discriminative for encrypted video traffic classification, confirming the value of flow-level statistics over raw-payload inspection. Zhao et al. [13] expose fundamental limitations of deep representation learning for encrypted traffic: models trained on per-packet data splits exploit shortcut features such as TCP sequence numbers and acknowledgment numbers, and their accuracy collapses below 40% when evaluated correctly on per-flow splits. Shallow models built on expert-selected statistical features (packet lengths, TCP window sizes, flow-level timing statistics) consistently match or outperform large Transformer- and Mamba-based encoders at a fraction of the computational cost. These findings confirm that handcrafted statistical vectors are a robust baseline for retrieval-based classification. At the same time, end-to-end learned representations, where a CNN or MLP backbone maps the raw per-packet sequence to a compact embedding, can capture non-linear inter-feature interactions that expert statistics may miss. We therefore evaluate both feature regimes in our experiments.

Closer to our retrieval paradigm, Shen et al. [14] apply a weighted k-NN approach with self-adaptive feature importance to fine-grained encrypted traffic classification using statistical flow features. However, this method does not address new-class adaptation, OOD detection, or indexing scalability. The new-class adaptation gap, in particular, motivates the few-shot

approaches surveyed next.

Recent work has explored few-shot and metric learning to reduce the labeled-data burden in encrypted traffic classification. Bovenzi et al. [15] evaluate multiple paradigms on rare mobile-app traffic: transfer-learning variants (fine-tuning and head-only freezing) require gradient updates to backbone or classification-head parameters for each new class. Li et al. [16] propose CL-MetaFlow, a two-stage training procedure (supervised contrastive pre-training followed by meta-learning fine-tuning) whose encoder is then frozen. New classes are handled by computing per-class prototype vectors from as few as one labeled sample, with no further gradient steps. Zhao et al. [17] freeze a large pre-trained backbone but require retraining a lightweight contrastive module (20 gradient-step epochs, under 15 seconds) whenever new classes emerge. Crucially, this refresh requires retaining or reusing base-class samples to maintain inter-class separation in the contrastive space.

Our system differs fundamentally from all of the above (see Section 1.1): the feature backbone is trained *once* on the base classes and then frozen permanently, while new classes, even from a single labeled sample, are absorbed by inserting their individual embeddings directly into the ANN index in $\mathcal{O}(\log n)$ time, with no gradient steps, no access to prior training data, and no per-class centroid recomputation.

A related but distinct challenge arises when a query comes from a class entirely outside the known distribution, with no samples available. OOD detection has received increasing attention as a complement to closed-world classifiers [19, 20]. Ansari et al. [21] propose synthesizing in-distribution samples with added noise and training an “Oracle” model that treats OOD

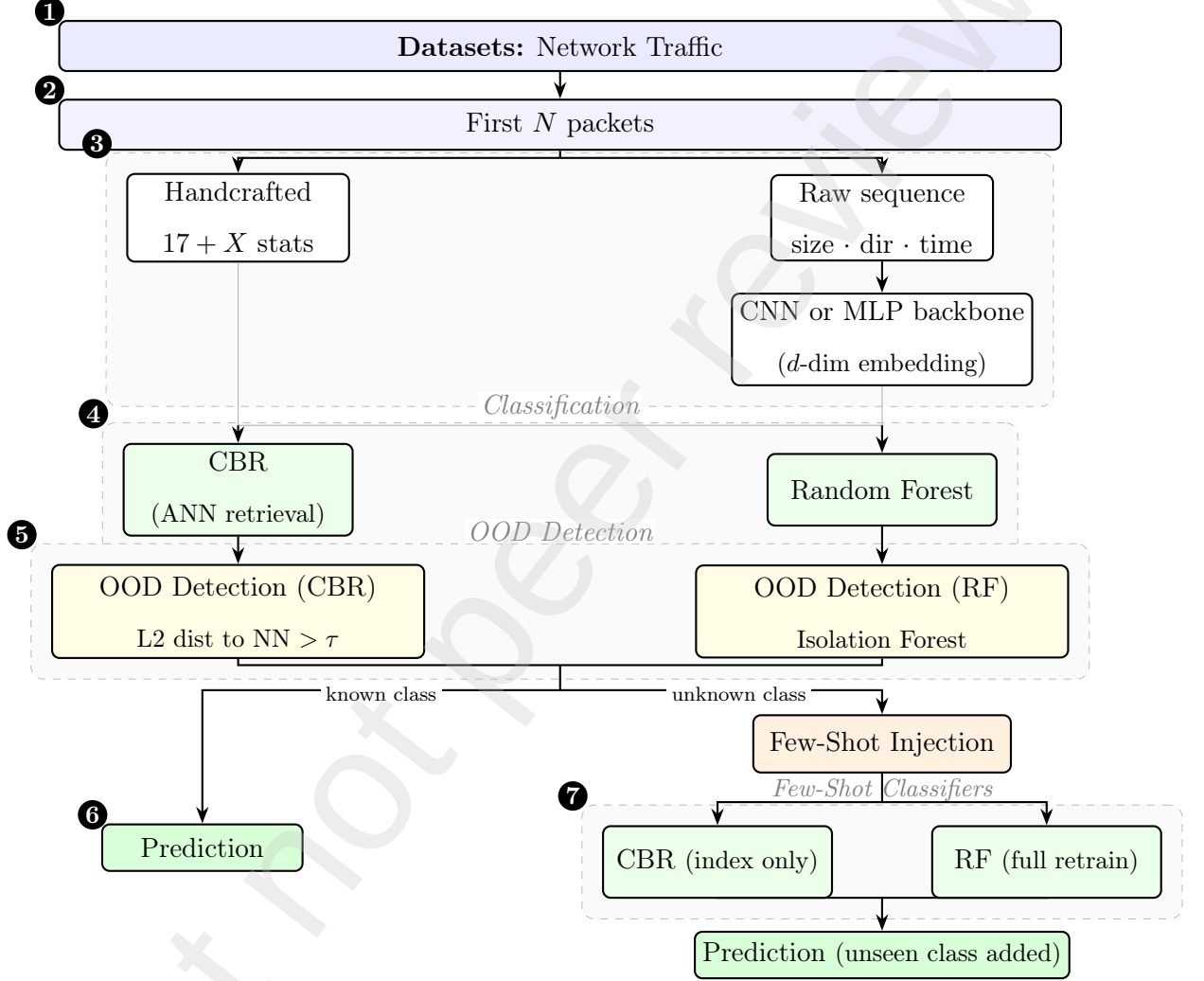


Figure 1: Overview of the full experiment pipeline. Every combination of dataset, feature representation, and classifier shares the same OOD detection and optional few-shot injection stages. The gray bus bar indicates that both feature paths (handcrafted statistics and deep embedding) feed both classifiers (CBR and RF), where CBR denotes Classification By Retrieval via ANN search over the HNSW index. CBR OOD uses a global L2 distance threshold τ , while RF OOD uses Isolation Forest [18].

as an explicit $(K+1)$ -th class, enabling standard tabular classifiers to handle open-set inputs. Zhao et al. [17] compute a per-class OOD threshold from the class centroid and variance in a contrastive embedding space. This approach requires maintaining per-class statistics and relies on a Gaussian distributional assumption that may not hold in practice, with computational overhead that grows with the number of classes. In contrast, our approach applies a single global ANN distance threshold that is distribution-free and requires no class-level bookkeeping, while unifying classification and OOD detection within the same HNSW retrieval step and requiring no additional training for OOD identification[22].

3. Methodology

Figure 1 illustrates the complete pipeline described in this section. The proposed system processes raw network traffic (stage 1 in Figure 1) by organizing packets into bidirectional flows and extracting fixed-length feature vectors from the first n packets of each flow (stage 2). Two parallel learning branches then operate on these vectors (stage 3): (1) A HandCrafted (HC) branch that optionally re-weights features via a learned per-feature W -vector; (2) A deep learning branch that trains a CNN or MLP backbone to map packet sequences to compact embeddings.

Both branches feed a unified CBR classification stage (stage 4), where representations are indexed in an HNSW approximate nearest-neighbor structure, and each query is classified by retrieving its closest stored neighbor. This same retrieval step enables out-of-distribution detection (stage 5) by treating flows whose nearest-neighbor distance exceeds a global threshold as

unknown. To support early-classification constraints and real-world deployment scenarios where classification must occur with minimal observation, we restrict each flow to the first N packets. We evaluate two feature pipelines (Handcrafted feature pipeline, Deep learning backbone pipeline) and two downstream classifiers (ANN, RF), which together define the main comparison axes of this study.

A core premise of our research is that effective classification can be achieved using a uniform, protocol-agnostic feature set derived solely from the first N packets of a flow. We intentionally avoid protocol-specific parsing (such as extracting TLS Handshake extensions) to maintain robustness against encryption updates (e.g., ECH in TLS 1.3).

For each flow, defined by its 5-tuple, we extract two types of features that together constitute the input for our Handcrafted (HC) pipeline, as summarized in Table 1:

1. **Statistical features (contextual behavior):** Aggregated metrics including minimum, maximum, mean, and standard deviation of packet sizes, inter-arrival times, and direction, calculated bi-directionally.
2. **Sequential features (raw fingerprint):** The ordered sequence of signed packet sizes $(S_1, S_2, \dots, S_N)$, which preserves the shape of the handshake and initial data burst, and is critical for early identification.

By concatenating these two types of features, we construct a unified, protocol-agnostic feature vector of dimensionality $17 + N$. This vector serves as the foundational representation evaluated within our handcrafted framework.

Feature group	Slots in vector	Description
Packet sizes	N	Signed packet sizes (positive = client→server, negative = server→client)
Statistics of packet sizes	4	Min, max, mean, std of packet sizes
Size delta between packets	2	Mean and std of inter-packet size differences
Packets per second	2	Packets per second in each direction (forward and backward)
Inter-arrival time	9	Min, max, and mean of bidirectional, forward, and backward inter-arrival times

Table 1: Feature groups extracted per flow. Total dimensionality is $17 + N$ (9 inter-arrival + 4 size statistics + 2 size delta + 2 PPS + N signed packet sizes).

Handcrafted (HC) feature pipeline. This pipeline directly operates on the composite vector described above, which combines the 17 statistical metrics with the N sequential packet sizes (yielding a total of $17 + N$ dimensions; see Table 1). For the RF classifier, due to the fact that RF is inherently scale-invariant because tree splits are invariant to monotone per-feature transformations, it is trained on raw feature values with no normalization.

For the ANN classifier, we evaluate two normalization variants to process this combined vector:

- ANN-raw: Raw feature values are indexed directly into the HNSW structure without any preprocessing.
- ANN-norm (W-vector): A *LearnableScaler*, consisting of a single learned weight per feature, is trained with batch-hard triplet loss [23] to produce a class-discriminative re-weighting of the feature space. The scaled vector is then indexed. Unlike a deep backbone, this variant does not reduce dimensionality, but rather re-weights the existing $17 + N$ feature dimensions.

Deep learning backbone pipeline. In contrast to the handcrafted approach that relies on explicit vector concatenation, the deep learning branch bypasses manual feature engineering by learning representations directly from raw network attributes. A CNN or MLP backbone is trained on the raw per-packet sequence and maps each flow to a compact d -dimensional embedding.

Both architectures receive the same $3 \times N$ input matrix whose rows correspond to per-packet inter-arrival time, direction, and size. Because these three channels differ by several orders of magnitude, a per-channel batch normalization layer normalizes the input before any learned transformation. The backbone is trained separately for each packet budget N and is frozen permanently once training is complete, with only its weights retained for embedding extraction.

CNN backbone. The CNN is a stack of four one-dimensional residual blocks with dilations $[1, 2, 4, 1]$ and channel widths progressing as $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 128$. Each residual block applies a dilated convolution (kernel size 5) followed by a non-dilated convolution (kernel size 3), GELU activations, and batch normalization, with a learned 1×1 projection on the skip

path when the channel count changes. Dilation progressively expands the receptive field across the packet sequence without increasing parameter count. After the four blocks, adaptive average pooling collapses the temporal dimension to a single vector, which a linear layer then projects to d dimensions. The output is L2-normalized to lie on the unit hypersphere.

MLP backbone. The MLP flattens the $3 \times N$ input to a $3N$ -dimensional vector and applies batch normalization at the input. Three hidden layers of widths 128, 64, and 32 follow, each consisting of a linear projection, ReLU activation, layer normalization, and dropout. Layer normalization is preferred over batch normalization in the hidden layers because the balanced batch sampler (described below) varies per-batch class composition across epochs, making batch statistics unreliable within hidden layers. A final linear layer projects to d dimensions, and the output is L2-normalized.

Shared training procedure. Both backbones are trained with the same procedure (stage 4). A cosine classification head with a learnable temperature scalar s (initialized to 10) computes logits as $s \cdot (\mathbf{z} \cdot \hat{W}^\top)$, where $\mathbf{z}$ is the L2-normalized embedding and $\hat{W}$ is the row-normalized weight matrix. Training minimizes a joint loss combining cross-entropy and Supervised Contrastive loss [24] with a tuned SupCon weight temperature 0.1. Mini-batches are constructed with a balanced batch sampler drawing $P=32$ classes with $K=4$ samples each, yielding an effective batch size of 128 and ensuring every batch contains sufficient within-class pairs for the contrastive term. The optimizer is AdamW with learning rate 5×10^{-4} and weight decay 10^{-4} . The learning rate follows a three-epoch linear warmup followed by cosine annealing for the remaining epochs. Training stops early when validation accuracy does

not improve for 10 consecutive epochs, and the checkpoint with the highest validation accuracy is retained. After training, the classifier head is discarded and the backbone weights are frozen for all downstream use.

Downstream classifiers. Both pipelines are evaluated with two classifiers: RF, a standard ensemble classifier trained on labeled feature vectors, used as a supervised baseline. The second is CBR, which implements ANN search over an HNSW vector index (Elasticsearch [25]). CBR performs no discriminative training at inference time: every query is answered by finding its closest match in the index. We use the term CBR throughout the paper to emphasize this retrieval-only property. CBR is the focus of our efficiency and adaptability analysis.

Normalization is applied selectively based on the classifier. RF uses raw feature values in all cases: normalization does not improve tree-based classifiers, and omitting it ensures that normalization overhead is not charged against RF in runtime comparisons. For ANN on handcrafted features, two regimes are compared: ANN-raw (no preprocessing) and ANN-norm (W-vector learned scaling, described above). Deep-backbone training requires no additional preprocessing beyond assembling the $3 \times N$ per-packet matrix; a separate backbone model is trained for each packet budget N .

3.1. Classification By Retrieval - CBR

CBR classifies each query vector by identifying its closest counterparts within a pre-indexed set of labeled reference vectors. Unlike parametric classifiers (e.g., RF or traditional neural networks) that require computationally expensive retraining cycles to accommodate unseen classes, CBR bypasses inference-time model updating entirely. This retrieval step is realized through

ANN search via the HNSW algorithm, which keeps it scalable by achieving $\mathcal{O}(\log n)$ query time. By leveraging an ANN architecture, adding a new class to the system simplifies to a pure indexing operation—injecting new labeled vectors into the existing space—allowing for zero-downtime, few-shot adaptation.

HNSW builds a multi-layered proximity graph over the indexed vectors. During search, traversal begins at the coarsest layer and progressively narrows toward the nearest neighbor, achieving $\mathcal{O}(\log n)$ average query time without exhaustive comparison. The graph topology is controlled by connectivity parameter m and construction beam width $ef_construction$, which govern the index quality–build cost trade-off. The specific values used in our evaluation are given in Section 4.

Classification and anomaly detection are unified in this framework. When a query vector is submitted, its nearest indexed neighbor determines the predicted label. If the distance to that neighbor exceeds a threshold τ , the sample is flagged as out-of-distribution (see Section 3.3). Adding a new class requires only indexing its representative vectors, with no retraining or model recalibration needed, making the system directly suitable for few-shot class injection.

3.2. Few-shot adaptation pipeline

Each type of training follows the same staged pipeline, illustrated with numbered stages (1-6) in Figure 1:

1. **Backbone training (optional):** Train a feature extractor on a set of base (seen) classes. After training, the backbone is frozen and used to generate embeddings for indexing and adaptation.

2. **Indexing:** Build reference representations for the currently known classes. In embedding-based variants, references are built in embedding space; in classic variants, references may be built directly in feature space.
3. **Few-shot class injection:** Introduce K labeled samples per unseen class into the system, which are absorbed by the few-shot classifiers (stage 7 in Figure 1).
4. **Adaptation to unseen classes:** A primary comparison axis of this work is *how each classifier incorporates unseen classes* given only K labeled examples.
 - **ANN:** The K few-shot vectors are inserted directly into the existing HNSW index. No model parameters are updated; the new class becomes queryable immediately at $\mathcal{O}(K)$ indexing cost.
 - **RF:** The model must be retrained from scratch on the combined dataset (all base training samples $\cup$ the K few-shot samples per unseen class). Decision trees cannot be incrementally updated without a full structural rebuild.

This asymmetry is central to our argument: in dynamic network environments where new application or malware classes emerge continuously, ANN can absorb new classes at negligible cost, while RF requires full retraining proportional to the entire training set size.

5. **Evaluation:** Evaluate classification outcomes for known classes (true vs. false predictions; stage 6 in Figure 1). When enabled, also evaluate OOD detection outcomes.

3.3. Out-of-distribution detection

ANN-based OOD. OOD detection in the ANN pipeline is a natural byproduct of the ANN retrieval step. A query is flagged as OOD if its L2 distance to the nearest indexed neighbor exceeds a global threshold τ :

$$\text{OOD}_{\text{ANN}}(x) = \mathbb{I} \left[\min_{c \in \mathcal{C}} d(z(x), r_c) > \tau \right], \quad (1)$$

where $z(x)$ is the feature (or embedding) representation of sample x , r_c is the nearest indexed vector of class c , and $d(\cdot, \cdot)$ is the ℓ_2 distance between representations. No additional model or computation is required beyond the search already performed for classification.

RF-based OOD. RF does not produce distances in a metric space, so a distance-based rule cannot be applied directly. Instead, we use **Isolation Forest** [18] as a standalone anomaly detector alongside the RF classifier. Isolation Forest builds an ensemble of random decision trees that recursively partition the feature space with random cuts; data points that are isolated in fewer splits—because they lie in sparse, atypical regions—receive a high anomaly score and are flagged as OOD. The detector is trained once on the base training set (in-distribution data) and applied at test time independently of the RF classifier. Isolation Forest is well-suited to this setting: it is non-parametric, makes no distributional assumptions, and scales efficiently to the high-dimensional handcrafted feature vectors used in our pipeline.

4. Experimental Design

We evaluate few-shot adaptation for encrypted-traffic classification across multiple datasets with different properties and label spaces. Unless noted

otherwise, each dataset is treated as a multi-class classification problem using the most relevant label level for that dataset.

4.1. Datasets

We conduct independent experiments on four datasets, each representing a distinct network environment. Each dataset is used separately. No merging or cross-dataset training is performed. This diversity demonstrates that our feature set is agnostic to the specific application type or protocol version.

- **CESNET-TLS22** [26] is a large-scale benchmark for encrypted traffic classification collected from the CESNET2 optical backbone. The full collection spans approximately two weeks and contains roughly 141 million flows across 191 application classes, labeled using TLS Server Name Indication (SNI). The dataset combines standard flow statistics with detailed packet metadata for the first 30 packets. In our experiments we use a single day of traffic from this collection. After excluding classes with fewer than 100 samples, our subset comprises 187 application classes across 21 service categories, totaling 15,200,175 flows. The per-class distribution is shown in Figure 3. The few-shot classes used to evaluate open-world adaptation are listed in Table 2a.
- **Cellular Network Traffic (Radcom)**: A proprietary dataset of encrypted cellular network traffic provided by Radcom Ltd. [27], a network analytics company that specializes in mobile network analysis. The data was collected from a live production cellular network under operational conditions, capturing real subscriber sessions. The dataset

covers a diverse range of mobile application categories. Due to subscriber privacy constraints and commercial sensitivity, this dataset is not publicly available. Its use in this study was conducted under a data-sharing agreement, with all traffic anonymized at the point of collection. After excluding classes with fewer than 100 samples, the dataset comprises 35 application categories and 13,893 flows in total, with per-class counts ranging from 105 to 1,101. Of these 35 classes, 13 are held out as unseen classes for few-shot evaluation and 22 are candidates for the main retrieval index. The retrieval-index experiments apply a stricter threshold of 300 samples, which excludes 6 of these candidates (each with 217 to 290 samples), so the base retrieval index comprises 16 application classes. The class distribution is shown in Figure 2. The few-shot classes are listed in Table 2b.

(a) CESNET-TLS22 few-shot classes.

Application	Flows
eidentita	920
cesnet-perun	742
justice-isir	169
xiaomi-tracking	84
adobe-search	81
redmine	74
ksrzi	12
apple-siri	8
sumava-camdata	6
Total	2,096

(b) Radcom few-shot classes.

Application	Flows
gcm	214
accuweather	202
itunes	200
vkontakte	189
mailru	175
apple_update	171
twitter	158
android_cnxmgr	141
bytedance	118
google_accounts	114
facebook_messenger	109
twitch	108
criteo	105
Total	2,004

Table 2: Few-shot classes per dataset ($K \in \{5, 10, 20, 30, 50, 100\}$). **Bold:** ≥ 100 samples, participates in all K -shot experiments. Non-bold: participates only when $K \leq$ sample count.

4.2. Experimental parameters

All experiments are run across the following parameter grids.

Packet budget: Each flow is truncated to the first N packets, with $N \in \{5, 10, 15, 20, 25, 30\}$. All experiments are repeated at each budget to quantify the impact of limited observation length.

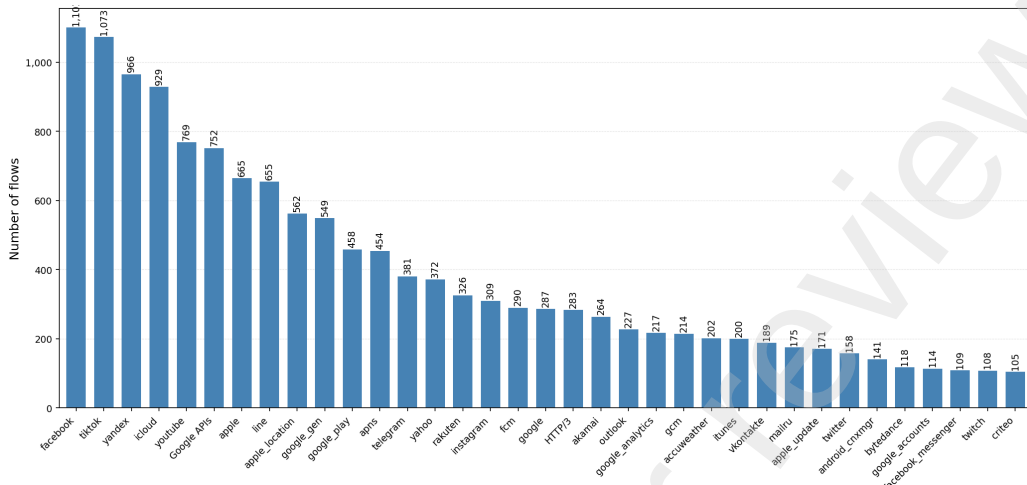


Figure 2: Class distribution across all splits of the Radcom dataset (35 classes shown: 22 main-set candidates plus the 13 few-shot classes, with classes below 100 samples excluded). After the 300-sample retrieval-index threshold, 16 of the 22 candidates form the base index.

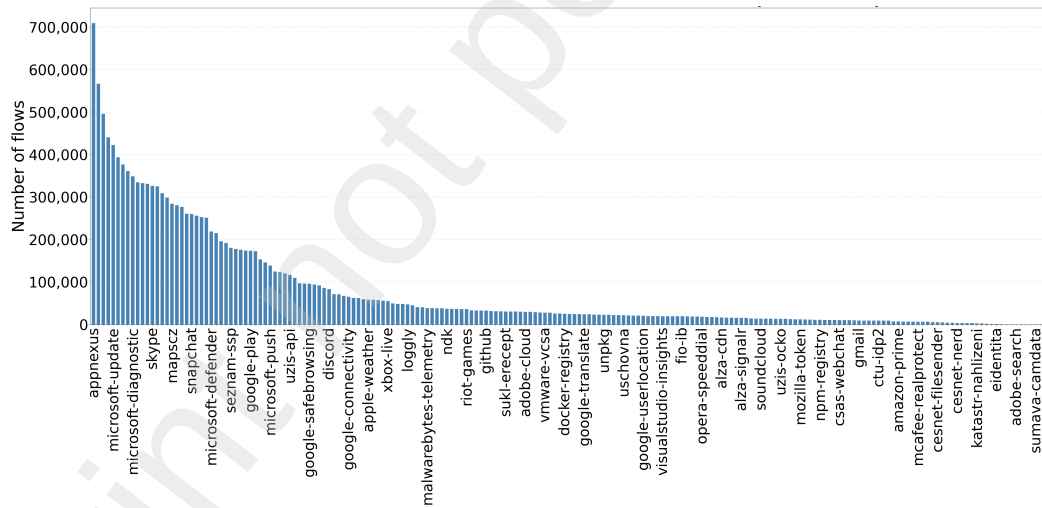


Figure 3: Class distribution across all splits, CESNET-TLS22 dataset (187 classes, 15,200,175 total flows; one day of traffic from the full two-week collection).

Few-shot budget: Unseen classes are introduced with up to K labeled samples each, with:

$$K \in \{5, 10, 15, 20, 25, 30, 50, 100\}. \quad (2)$$

Sampling is intended to be close to uniform across unseen classes, subject to availability.

A class can participate in a K -shot experiment only if it contains at least K labeled examples. As K increases, fewer classes meet this requirement, making direct comparison across K values difficult. To address this, we evaluate two complementary regimes. In the **expanding-class regime**, each K -shot experiment uses all few-shot classes with at least K samples, so the evaluated class set grows as K decreases. In the **fixed-class regime**, all K values are evaluated on the same subset, namely classes with at least 100 samples (the maximum K in our grid), ensuring results are directly comparable across K . Table 2a lists all few-shot classes with at least 5 samples (the minimum K); classes that participate in all experiments are shown in bold.

Embedding dimensions (Experiment 2): The deep backbone is trained to produce d -dimensional embeddings, with $d \in \{16, 32, 64, 128\}$. We trained backbones for all of these dimensions, but report results only for $d=128$.

Elasticsearch index configuration: Vectors are stored as `dense_vector` fields in Elasticsearch, indexed with HNSW using $m=16$ and `ef_construction=100`, and ℓ_2 -norm (`l2_norm`) similarity. At query time, `num_candidates=10` for standard classification and `num_candidates=100` for OOD detection, trading retrieval accuracy against latency. The score returned by Elasticsearch

is related to the ℓ_2 distance by $score = 1/(1 + d^2)$, so the distance used for OOD thresholding is recovered as $d = \sqrt{(1/score) - 1}$.

OOD threshold evaluation: Rather than calibrating a single threshold value, ANN-based OOD detection is evaluated across a predefined grid of thresholds applied to the Elasticsearch score. The grid comprises 17 score thresholds:

$$\begin{aligned} \mathcal{T} = \{ & 1.0, 0.5, 0.2, 0.1, 0.05, 0.005, 0.001, \\ & 8 \times 10^{-4}, 7 \times 10^{-4}, 6 \times 10^{-4}, 5 \times 10^{-4}, 4 \times 10^{-4}, \\ & 3 \times 10^{-4}, 2 \times 10^{-4}, 10^{-4}, 5 \times 10^{-5}, 10^{-5} \}. \end{aligned}$$

Because $score = 1/(1 + d^2)$, each score threshold directly corresponds to a fixed ℓ_2 distance threshold τ . AUROC is used as the primary OOD metric and is threshold-free; per-threshold results characterise the system’s operating points across the sensitivity–specificity trade-off.

4.3. Experiments

Experiment 1: Handcrafted features, ANN vs. RF. **Goal:** Establish a strong handcrafted baseline and compare ANN and RF across all datasets in terms of classification accuracy, OOD detection, and few-shot adaptation speed. **Datasets:** CESNET-TLS22, Radcom. For each dataset and each packet budget N , we extract the $17+N$ handcrafted feature vector (Table 1). RF is trained on raw features; ANN is evaluated in two configurations (ANN-raw and ANN-norm; see Section 3). OOD detection is applied (distance threshold for ANN, Isolation Forest for RF) and few-shot class injection is performed with $K \in \{5, 10, 20, 30, 50, 100\}$ samples per unseen class.

Experiment 2: Deep backbone — in-domain evaluation. **Goal:** Evaluate whether a learned embedding backbone improves classification accuracy, OOD detection, and few-shot efficiency compared to the handcrafted baseline (Experiment 1), using the same datasets and the same downstream pipeline. **Datasets:** CESNET-TLS22, Radcom. For each dataset, a CNN or MLP backbone is trained on a subset of base classes from that dataset using the $3 \times N$ raw-sequence input (size · direction · time), producing d -dimensional embeddings ($d \in \{16, 32, 64, 128\}$). A separate backbone is trained for each dataset–packet-budget combination ($N \in \{5, 10, 15, 20, 25, 30\}$). After training, each backbone is **frozen permanently**—it is never updated, fine-tuned, or modified at any subsequent stage of the experiment. The frozen backbone embeds all flows, and the downstream pipeline from Experiment 1 is then applied identically:

1. **Classification:** ANN retrieval over the HNSW embedding index, or RF trained on embeddings.
2. **OOD detection:** ANN uses the L2 distance threshold τ . RF uses Isolation Forest (contamination set to `auto`, letting scikit-learn estimate the inlier fraction from the training data)
3. **Few-shot injection:** Unseen classes are introduced with $K \in \{5, 10, 20, 30, 50, 100\}$ labeled samples. ANN indexes the new embeddings directly; RF re-trains on the combined embedding set. The backbone remains frozen in both cases.

Results are compared directly against Experiment 1 to isolate the contribution of the learned embedding space over the handcrafted features.

4.4. Metrics and efficiency

We report accuracy, macro-averaged precision, recall, and F1-score for all classification experiments. Because both datasets are class-imbalanced, we use macro averaging throughout: metrics are computed per class and then averaged equally across all classes, giving the same weight to rare and frequent classes alike. Where applicable, runtime accounting includes normalization time (for ANN experiments) and can be decomposed by pipeline stage (feature extraction or embedding, indexing, retraining, inference).

Hardware and software environment. All experiments were run on a single workstation to ensure that the reported runtimes are directly comparable across methods. The machine is equipped with two Intel Xeon Silver 4410Y CPUs (24 physical cores, 48 threads in total), 251 GB of RAM, and three NVIDIA RTX A4500 GPUs (20 GB each); deep-learning backbones were trained on a single GPU. The software stack is Ubuntu 22.04.5 LTS (Linux kernel 5.15), Python 3.11, PyTorch 2.6.0 with CUDA 12.4, scikit-learn 1.7.1 (Random Forest and Isolation Forest baselines), and Elasticsearch 9.1.0 for HNSW indexing and retrieval. Unless otherwise stated, all runtime measurements were obtained on this identical configuration.

5. Results

This section reports experimental results on the CESNET and RADCOM datasets, comparing CBR (Elastic HNSW) against RF across four evaluation dimensions: base classification performance, temporal dynamics, few-shot adaptation, and out-of-distribution detection. Throughout this section we discuss the primary performance metric, the macro-averaged F1-score, to

ensure rigorous evaluation across the imbalanced class distributions of both datasets. An exhaustive evaluation that additionally reports Accuracy, Precision, Recall, and Micro F1 is provided in the supplementary tables. All comprehensive tabular data, intermediate packet configurations, and complete metric profiles are available in our anonymized supplementary results repository [28]. The full source code required to reproduce these results will be released publicly upon acceptance. Training time t (seconds) measures the time to build the CBR index or fit the RF model. Feature extraction time is identical for both methods and is excluded from t . Results from both Experiment 1 (handcrafted features; Section 4) and Experiment 2 (CNN and MLP backbones; Section 4) are reported jointly throughout each subsection. CESNET serves as the primary dataset for establishing and explaining results; RADCOM results validate robustness and highlight dataset-specific differences.

5.1. Base Performance and Training Efficiency

This subsection covers Experiments 1 and 2 jointly: HC features (Experiment 1) establish the baseline, and CNN/MLP embeddings (Experiment 2) are compared against it and against the RF baseline across both datasets. The evaluation on CESNET reveals a consistent pattern across all backbones: RF achieves higher classification accuracy than CBR, though the magnitude of this gap varies substantially with the choice of feature representation. Among all configurations, the CNN backbone delivers the strongest CBR performance, followed by MLP embeddings and then HC features, establishing a $\text{CNN} > \text{MLP} > \text{HC}$ ranking for retrieval. This ordering does not carry over to RF, where handcrafted features become the most competitive and

MLP trails the other two backbones.

In terms of absolute accuracy, CNN-CBR and CNN-RF on CESNET produce nearly identical macro F1 scores, averaging 0.923 and 0.925 respectively across all packet-count configurations (Figure 4a). This near-parity indicates that CBR operates at the frontier of what RF can achieve when using deep convolutional features, making the two approaches practically interchangeable in terms of predictive quality on this dataset. The MLP backbone is nearly on par between the two classifiers (0.871 for CBR vs. 0.876 for RF), while handcrafted features exhibit the largest gap of all: HC-RF outperforms HC-CBR by an average margin of 0.133 F1 points. This pronounced gap for HC features reflects two compounding effects.

The raw feature geometry produced by handcrafted statistics does not lend itself naturally to nearest-neighbour retrieval, whereas tree-based ensembles can construct more discriminative decision boundaries in that space. Furthermore, with `num_candidates=10`, each CBR query explores only a small region of the index; in the handcrafted feature space, where class boundaries overlap more, a small candidate pool increases the chance of returning a wrong nearest neighbour. In the deep-backbone pipelines, contrastive training clusters same-class embeddings tightly so that 10 candidates suffice for correct retrieval.

CBR is dramatically faster to train than RF across every backbone, as Figure 5 makes visually explicit. On CESNET, CNN-CBR requires approximately 133 seconds versus 1,270 seconds for CNN-RF, a roughly 10 $\times$ speedup. For MLP embeddings, the advantage grows to nearly 50 $\times$ (146 seconds vs. 7,400 seconds), and for HC features CBR trains in 44 seconds

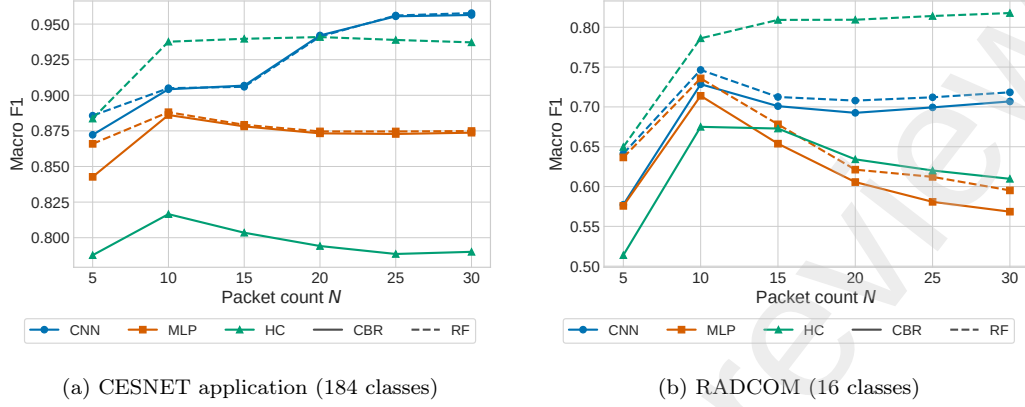


Figure 4: Base classification performance: macro F1 as a function of the observed packet count N for CBR (Elastic HNSW, solid) and RF (dashed) across the CNN, MLP, and HC backbones (colour and marker per backbone). For CBR the $\text{CNN} > \text{MLP} > \text{HC}$ ranking holds across packet counts, CNN-CBR and CNN-RF are near-identical on CESNET, and CBR trains roughly 10–50 $\times$ faster than RF (training times reported in the text).

against 1,926 seconds for RF. The gap widens dramatically in the few-shot regime (Figure 5b), where injecting $K=5$ vectors into the index makes CBR two to three orders of magnitude faster than full RF retraining.

The RADCOM dataset (Figure 4b) validates the accuracy and speed trends observed on CESNET, with one notable nuance. The HC feature gap is in fact slightly larger on RADCOM (0.160 average margin across packet counts), reinforcing the conclusion that handcrafted features are structurally disadvantaged in a retrieval setting. Training efficiency is fully replicated: CBR consistently trains 6 to 15 times faster than RF depending on the backbone. However, the relationship between CNN-CBR and CNN-RF diverges quantitatively across the two datasets. On CESNET the two are essentially tied, whereas on RADCOM, RF leads CNN-CBR by 1 to 6 percentage points across packet-count configurations. This divergence suggests that

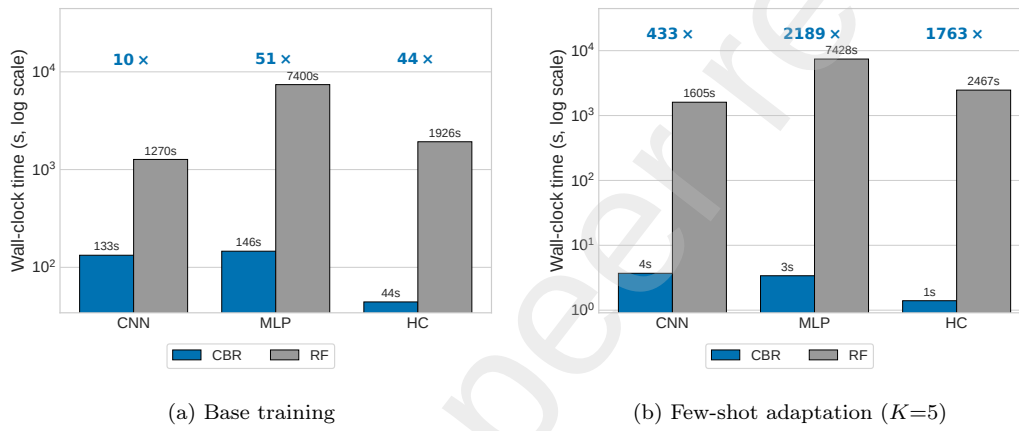


Figure 5: Training and adaptation efficiency on CESNET application classification (184 classes), CBR (Elastic HNSW) versus RF, on a logarithmic time axis (mean over the six packet counts N). (a) base training time, where CBR is roughly 10 $\times$ (CNN), 51 $\times$ (MLP), and 44 $\times$ (HC) faster than RF. (b) few-shot adaptation time at $K=5$, where injecting new vectors into the index makes CBR two to three orders of magnitude faster than full RF retraining (433–2,189 $\times$). The bold factor above each backbone group is the RF/GBR speedup.

CBR’s competitiveness scales with dataset complexity: on a large and diverse dataset such as CESNET with 184 application classes, the deep embedding space is rich enough that retrieval-based classification matches discriminative learning, whereas on the smaller RADCOM 16-class subset the reduced task complexity gives RF more room to exploit its ensemble structure.

5.2. Temporal Dynamics

Across both datasets, the most substantial accuracy improvement occurs when the number of observed packets per flow increases from 5 to 10. On CESNET, CNN-CBR F1 rises from 0.872 at $N=5$ to 0.904 at $N=10$, an improvement of over 3 percentage points (Figure 4a). A comparable leap appears on RADCOM, where CNN-CBR climbs from 0.577 to 0.729 over the same transition (Figure 4b). This pattern reflects the fundamental informativeness of early packet payloads: a single initial packet is often insufficient to discriminate encrypted application traffic reliably, but capturing the first ten packets provides enough behavioural signal to characterise the flow with high confidence.

Beyond $N=10$, the two datasets diverge in their trajectories. On CESNET, accuracy continues to improve monotonically as the packet count rises from 10 to 30, with CNN-CBR reaching 0.956 at $N=30$. This steady gain indicates that longer flows on CESNET carry progressively more discriminative content, rewarding models that observe more of the traffic sequence. On RADCOM, by contrast, performance plateaus after $N=10$ and decreases slightly at higher packet counts in several configurations. This saturation likely reflects a structural property of RADCOM flows: mobile application traffic tends to produce shorter and more stereotyped exchanges, so packets

beyond the initial burst introduce noise rather than additional signal. The finding implies that the optimal N should be selected based on the characteristics of the target network environment rather than set to a universal maximum.

The MLP backbone follows the same trend as CNN on CESNET and shows no degradation at short flow lengths. MLP-CBR reaches 0.843 at $N=5$ and stays within 0.873 to 0.886 across the remaining packet counts, closely tracking MLP-RF (0.866 at $N=5$). The CNN advantage over MLP therefore remains modest throughout, consistent with both backbones learning a metrically meaningful embedding even when only the first five packets are observed.

5.3. Few-Shot Adaptation

Each few-shot experiment injects K labelled samples per unseen class into a base model trained on the original class set. Macro F1 is computed on the unseen-class test flows only: including base-class flows would dilute the score because the unseen classes contribute only a small fraction of the test set. For CBR, injecting K samples is a constant-time index update with no retraining. For RF, the classifier is retrained from scratch on the base training set combined with the K new samples. Each table includes two footer rows showing representative adaptation time at $K=5$: CBR total adaptation time and RF full retraining time.

A consistent and prominent finding emerges: few-shot macro F1 (Figures 6 and 7) improves monotonically as K increases across all backbones and both datasets. On CESNET with a CNN backbone and $N=20$ packets, CBR F1 rises from 0.592 at $K=5$ to 0.841 at $K=100$, while RF F1 improves

from 0.363 to 0.673 over the same range. Both methods benefit from richer support sets, but the rate and absolute level of improvement differ meaningfully between them.

For the CNN backbone, CBR consistently outperforms RF in the few-shot regime on CESNET, achieving on average a macro F1 of 0.725 against 0.572 for RF across all K and packet-count combinations. This advantage is intuitive: a retrieval system can immediately incorporate new examples into its case base without any retraining, making it structurally well-suited to few-shot extension. RF must rebuild its ensemble to accommodate unseen classes, and with few examples per unseen class its generalisation is limited by the scarcity of the support. The speedup mirrors the base-training case: inserting K new vectors into an HNSW index requires $\mathcal{O}(K \log n)$ operations, whereas RF retraining from scratch is $\mathcal{O}((n+K) \cdot d \cdot T)$, where n is the training-set size, d is the feature dimension, and T is the number of trees. For MLP embeddings, CBR retains a smaller but consistent advantage on CESNET (0.655 vs. 0.582). Handcrafted features reverse the relationship: HC-RF achieves 0.723 compared to HC-CBR at 0.570, mirroring the base-performance pattern and reinforcing that the HC feature space is more effectively exploited by discriminative ensemble methods in both full-data and few-shot regimes.

Two class-set variants are reported for CESNET. The *varying* variant includes all unseen classes with at least K samples, so the class count shrinks as K grows (9 classes at $K=5$, 3 at $K=100$). The *fixed* variant evaluates only the 3 unseen classes present at all K values.

5.3.1. CESNET Application Classification

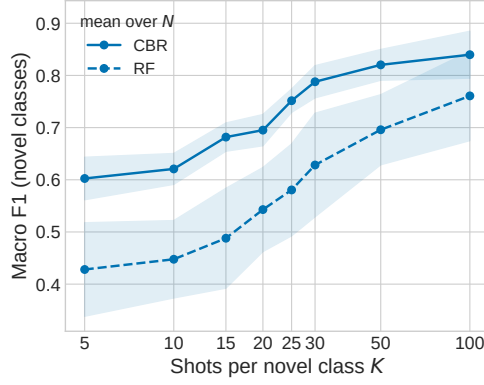
Figure 6 presents few-shot macro F1 for CNN, MLP, and HC pipelines under both class-set variants. Across all backbones, CBR adaptation requires only 2–5 seconds while RF retraining takes 1,000–2,500 seconds on CESNET, reflecting the fundamental complexity difference between HNSW index insertion and full ensemble retraining.

5.3.2. RADCOM Application Classification

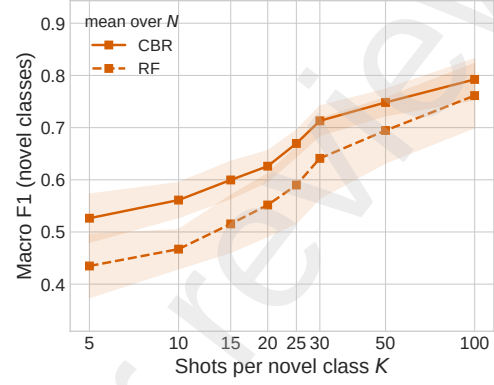
The RADCOM dataset (Figure 7) validates all three few-shot dynamics established on CESNET. CNN-CBR leads CNN-RF in few-shot classification (average F1 0.392 vs. 0.382 across all K and N), and HC-RF outperforms HC-CBR (0.531 vs. 0.435) across all N and K values. The absolute advantage for CNN-CBR is smaller on RADCOM than on CESNET, consistent with the lower overall complexity of the 16-class task. MLP few-shot performance is effectively tied between CBR and RF on RADCOM (0.352 vs. 0.354), compared to CBR’s clearer advantage on CESNET, a minor divergence that reflects the reduced task complexity rather than a structural change in behaviour. CBR adaptation requires 1–7 seconds for deep backbones and 2 seconds for HC, while RF retraining requires 64–130 seconds.

5.4. Out-of-distribution Detection

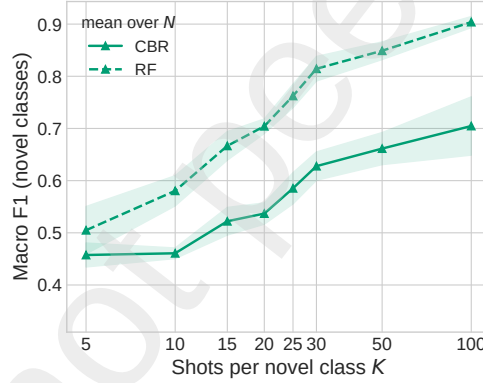
OOD detection is evaluated as binary AUROC: in-distribution samples are drawn from the base test set, while OOD samples are test flows from the unseen classes. AUROC is a threshold-free metric that summarises discriminative performance across all possible operating points without requiring the selection of a detection threshold. CBR uses L2 distance to the nearest



(a) CNN

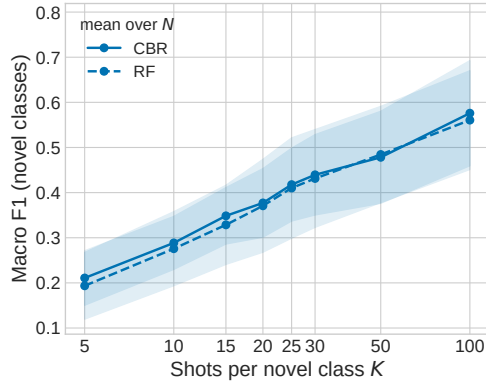


(b) MLP

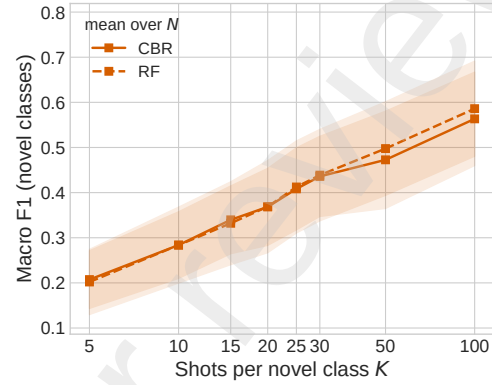


(c) HC

Figure 6: CESNET application few-shot adaptation: macro F1 on the unseen classes (varying class set) as a function of the per-class shot budget K , for CBR (index insertion only, solid) and RF (full retraining, dashed) across the CNN, MLP, and HC backbones. Each curve is the mean across the six packet counts N and the shaded band is one standard deviation. F1 rises monotonically with K . CNN-CBR leads CNN-RF, MLP-CBR holds a smaller lead, and HC-RF reverses the relationship over HC-CBR. CBR adaptation takes 2–5 s versus 1,000–2,500 s for RF retraining.



(a) CNN



(b) MLP



(c) HC

Figure 7: RADCOM application few-shot adaptation: macro F1 on the unseen classes (varying class set) as a function of the per-class shot budget K , for CBR (solid) and RF (dashed) across the CNN, MLP, and HC backbones. Each curve is the mean across the six packet counts N and the shaded band is one standard deviation. The trends mirror CESNET at lower absolute F1: CNN-CBR edges CNN-RF, MLP is effectively tied, and HC-RF leads HC-CBR. CBR adaptation takes 1–7 s versus 64–130 s for RF retraining.

indexed neighbour as the anomaly score (lower similarity indicates a likely OOD sample). RF uses one minus the maximum class probability as a proxy anomaly score.

The backbone ranking established in the classification experiments largely carries over to OOD detection. For CBR-based OOD detection on CESNET, the CNN backbone yields the highest average AUROC (0.904), followed by MLP (0.889) and HC (0.791) (Figure 8). The strong CNN result indicates that the dense embedding space learned by the convolutional model produces well-separated clusters, allowing distance-based scores to identify OOD flows with high reliability. MLP ranks a close second, confirming that its learned embedding also separates in-distribution and out-of-distribution flows effectively across the full packet budget. HC features trail both learned backbones in CBR OOD detection, indicating that the handcrafted feature space encodes enough structural variation to flag fundamentally different traffic patterns yet supports retrieval-based anomaly scoring less effectively than learned embeddings.

The RF-based OOD results expose a sharper contrast. For CNN and MLP backbones, CBR achieves comparable or better AUROC than RF on CESNET: CNN-CBR reaches 0.932 at $N=5$ and maintains 0.844–0.941 across the full packet budget, exceeding CNN-RF (0.896–0.907) at most values. For HC features, however, RF substantially outperforms CBR, with HC-RF achieving 0.893–0.931 against HC-CBR at 0.758–0.861, a gap of over 11 points on average. This result mirrors the pattern observed in base accuracy and few-shot performance, confirming that HC features are more naturally exploited by tree-based methods in both in-distribution and out-of-distribution

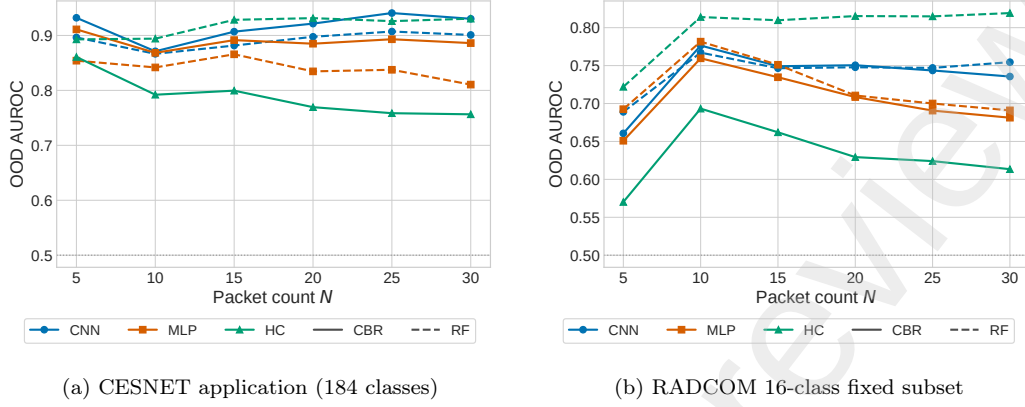


Figure 8: Out-of-distribution detection AUROC as a function of the observed packet count N for CBR (solid) and RF (dashed) across the CNN, MLP, and HC backbones (colour and marker per backbone). Higher is better, and the dotted line marks the 0.5 random-chance level. The CBR backbone ranking is $\text{CNN} > \text{MLP} > \text{HC}$, and HC-RF substantially outperforms HC-CBR on both datasets.

settings.

To characterise the sensitivity–specificity trade-off at a single operating point, Figure 8 is complemented by the Youden-optimal threshold for each backbone at $N=15$ (Tables 3a and 3b), i.e., the score below which a flow is flagged OOD that maximises sensitivity + specificity -1 (the ROC point farthest from the random-chance diagonal). *Score thresh.* is the Elasticsearch similarity score at that point, τ_{ℓ_2} is the equivalent ℓ_2 distance, *Sens.* is OOD recall (fraction of OOD flows correctly detected), and *Spec.* is base-class precision (fraction of in-distribution flows correctly retained).

(a) CESNET.					(b) RADCOM.				
Model	Score thresh.	τ_{ℓ_2}	Sens.	Spec.	Model	Score thresh.	τ_{ℓ_2}	Sens.	Spec.
CNN-CBR	0.962151	0.1983	93.6%	79.1%	CNN-CBR	0.995732	0.0655	92.4%	47.6%
MLP-CBR	0.994885	0.0717	85.1%	81.9%	MLP-CBR	0.999364	0.0252	89.0%	51.9%
HC-CBR	4.7×10^{-5}	146.1233	91.5%	63.2%	HC-CBR	5.2×10^{-5}	138.0433	87.4%	46.3%

Table 3: Youden-optimal OOD operating points at $N=15$. *Score thresh.* is the Elastic-search similarity score below which a flow is flagged OOD, τ_{ℓ_2} is the equivalent ℓ_2 distance, *Sens.* is OOD recall, and *Spec.* is base-class retention rate.

CNN-CBR and MLP-CBR reach balanced operating points, each exceeding 85% sensitivity and 79% specificity. The degenerate threshold for HC-CBR ($\tau = 146$) reflects that handcrafted features in $17+N$ dimensions span a far larger ℓ_2 scale than learned embeddings, so score distributions for in- and out-of-distribution flows overlap heavily and no effective threshold exists within the standard score range. The full ROC curves at $N=15$ are shown in Figure 9a: CNN-CBR and MLP-CBR trace well-separated curves reaching the upper-left quadrant, while HC-CBR collapses near the diagonal, consistent with the degenerate threshold in Table 3a.

The RADCOM dataset fully corroborates these OOD findings (Figure 8). The CBR backbone ranking remains $\text{CNN} > \text{MLP} > \text{HC}$, with average AU-ROC values of 0.730, 0.682, and 0.628 respectively. The HC RF advantage over HC CBR is even more pronounced on RADCOM: HC-RF reaches 0.722–0.819 against HC-CBR at 0.570–0.693, a gap of over 16 points on average. This widening of the HC deficit reinforces the conclusion that handcrafted statistical features support RF-based OOD detection far more effectively than retrieval-based methods, and that this structural incompatibility is a ro-

bust property of the HC feature representation rather than a dataset-specific artefact.

Table 3b shows the RADCOM Youden-optimal operating points. HC-CBR shows the same degenerate scale as on CESNET ($\tau > 100$), with the score distributions of in- and out-of-distribution flows largely overlapping.

CNN-CBR and MLP-CBR require very high score thresholds (score > 0.99 , $\tau < 0.07$), meaning that an OOD flow must be an almost-perfect nearest-neighbour match to avoid being flagged OOD. Sensitivity exceeds 87% for all three models, but specificity is approximately 46–48%, lower than CESNET because the smaller 16-class base set provides less discriminative margin between in- and out-of-distribution flows. As on CESNET, HC-CBR produces a degenerate threshold ($\tau = 138$) for the same reason: handcrafted feature vectors in $17+N$ dimensions span a far larger ℓ_2 scale than learned embeddings. The corresponding ROC curves for RADCOM (Figure 9b) confirm these patterns: CNN-CBR and MLP-CBR maintain clear separation from the diagonal, while HC-CBR again collapses near chance level.

6. Limitations

While the proposed retrieval framework is accurate and highly efficient, our study has several limitations.

Feature representation dependency. While we evaluated both handcrafted and learned feature pipelines, no single extraction method proved universally optimal across different classification paradigms. Learned deep embeddings (particularly CNNs) are naturally suited for distance-based retrieval and OOD detection, as they cluster effectively in metric space. Conversely,

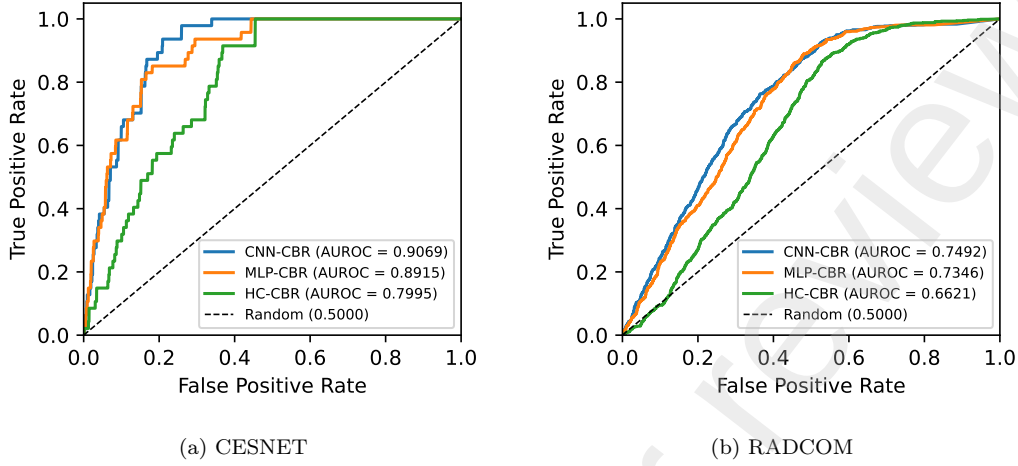


Figure 9: ROC curves for CBR OOD detection at $N=15$. AUROC values match Figure 8.

handcrafted statistical features remain highly competitive when paired with RF, but perform poorly under retrieval due to overlapping boundaries in the raw feature space. Consequently, a limitation of our framework—and the broader encrypted traffic analysis domain—is the strict dependency between the chosen feature geometry and the downstream classification algorithm.

Approximation introduces an accuracy trade-off. CBR relies on approximate search, so a limited candidate pool trades a small amount of accuracy for speed. This trade-off is most visible in the handcrafted regime and could be narrowed by enlarging the candidate pool at the cost of higher query latency.

Smaller class spaces narrow the advantage. On the 16-class RADCOM subset, RF remains slightly more competitive and OOD specificity drops to roughly 46 to 48 percent, because a smaller base set offers less discriminative margin between in- and out-of-distribution flows. CBR’s relative benefit is therefore most pronounced on large, diverse class spaces.

Evaluation scope. We evaluate one large public dataset and one proprietary cellular dataset, the latter of which cannot be shared. A broader study across more datasets, traffic domains, and explicit cross-dataset transfer remains future work.

7. Conclusion and Future Work

We presented an ultra-fast encrypted traffic classification system based on Classification By Retrieval. Each flow is represented by a fixed-length vector extracted from only the first few packets, indexed in an HNSW approximate nearest-neighbor structure, and classified by retrieving its nearest labeled neighbor, while out-of-distribution traffic is detected within the same step using a single global distance threshold. With CNN embeddings, CBR matches the RF baseline in accuracy (macro F1 of 0.923 versus 0.925 on CESNET-TLS22) while training roughly ten times faster, adapts to new classes two to three orders of magnitude faster, and detects out-of-distribution flows with up to 0.90 AUROC.

The practical advantage of retrieval-based classification grows with scale. When the number of classes is large and the total volume of labeled data is high, even if individual classes are not, parametric classifiers must pay a retraining cost that grows with the entire dataset, whereas CBR absorbs new data and new classes through indexing alone. In operational settings that demand fast and automatic adaptation, this asymmetry between cheap indexing and expensive retraining is a decisive and clearly felt benefit.

Several directions remain open. First, closing the handcrafted-feature gap through better metric learning or learned re-weighting, and studying the

candidate-pool size as an explicit accuracy-latency trade-off, would broaden the framework’s applicability. Second, hybrid representations that combine handcrafted statistics with learned embeddings may capture complementary signal. Third, cross-dataset and cross-domain transfer, adaptive or per-class OOD thresholds, and evaluation on streaming deployments under concept drift are natural next steps toward production use.

Acknowledgements

This work was supported by the Ariel Cyber Innovation Center, the Israel Innovation Authority, and the ENTM consortium. The work is patent-based on US Provisional Patent Application No. 63/448,460.

Data availability

The CESNET-TLS22 dataset is publicly available. The Radcom cellular-traffic dataset is proprietary and cannot be shared due to subscriber-privacy and commercial-sensitivity constraints. The code will be made publicly available upon acceptance.

References

- [1] Internet Security Research Group, Let's encrypt (2021).
URL <https://letsencrypt.org/about/>
- [2] Google, Quic, a multiplexed transport over udp (2023).
URL <https://www.chromium.org/quic/>
- [3] T. Böttger, F. Cuadrado, G. Antichi, E. L. Fernandes, G. Tyson, I. Castro, S. Uhlig, An empirical study of the cost of dns-over-https, in: Proceedings of the Internet Measurement Conference, 2019, pp. 15–21.
- [4] Ericsson, Ericsson mobility report, accessed: 2024-08-22 (2024).
URL <https://www.ericsson.com/en/reports-and-papers/mobility-report/dataforecasts/mobile-traffic-forecast>
- [5] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International conference on information networking (ICOIN), IEEE, 2017, pp. 712–717.
- [6] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE international conference on intelligence and security informatics (ISI), IEEE, 2017, pp. 43–48.
- [7] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference On Computer Communications, IEEE, 2019, pp. 1171–1179.

- [8] P. Wang, X. Chen, F. Ye, Z. Sun, A survey of techniques for mobile service encrypted traffic classification using deep learning, *Ieee Access* 7 (2019) 54024–54033.
- [9] V. F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, *IEEE Transactions on Information Forensics and Security* 13 (1) (2017) 63–78.
- [10] A. J. Pinheiro, J. d. M. Bezerra, C. A. Burgardt, D. R. Campelo, Identifying iot devices and events based on packet length from encrypted traffic, *Computer Communications* 144 (2019) 8–17.
- [11] M. Zhan, R. Zhao, X. Deng, Z. Xue, Q. Li, Z. Liu, G. Cheng, K. Xu, Flowrefiner: A robust traffic classification framework against label noise, *Advances in Neural Information Processing Systems* 38 (2026) 128242–128265.
- [12] A. Biernacki, Evaluating filter, wrapper, and embedded feature selection approaches for encrypted video traffic classification, *Electronics* 14 (18) (2025) 3587.
- [13] Y. Zhao, G. Dettori, M. Boffa, L. Vassio, M. Mellia, The sweet danger of sugar: Debunking representation learning for encrypted traffic classification, in: *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 296–310.
- [14] C. Ma, X. Du, L. Cao, Improved knn algorithm for fine-grained classification of encrypted network flow, *Electronics* 9 (2) (2020) 324.

- [15] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, A. Pescapé, Few shot learning approaches for classifying rare mobile-app encrypted traffic samples, in: IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2023, pp. 1–6.
- [16] Z. Li, J. Wang, Y.-F. Song, S.-H. Yue, Unlocking few-shot encrypted traffic classification: A contrastive-driven meta-learning approach, *Electronics* 14 (21) (2025) 4245.
- [17] L. Zhao, C. Wu, A. Yu, When pre-training meets contrast learning: Few-shot encrypted traffic classification with novelty detection, *IEEE Transactions on Networking* (2026).
- [18] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation forest, in: 2008 eighth IEEE international conference on data mining, IEEE, 2008, pp. 413–422.
- [19] S. Lu, Y. Wang, L. Sheng, L. He, A. Zheng, J. Liang, Out-of-distribution detection: A task-oriented survey of recent advances, *ACM Computing Surveys* 58 (2) (2025) 1–39.
- [20] T. Belkhouja, Y. Yan, J. R. Doppa, Out-of-distribution detection in time-series domain: A novel seasonal ratio scoring approach, *ACM Transactions on Intelligent Systems and Technology* 15 (1) (2023) 1–24.
- [21] A. Ansari, K. Wang, P. Xiong, Out-of-distribution aware classification for tabular data, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, 2024, pp. 65–75.

- [22] I. Meiri, R. Dubin, A. Dvir, C. Hajaj, Out-of-distribution is not magic: The clash between rejection rate and model success, in: 2025 20th Conference on Computer Science and Intelligence Systems (FedCSIS), IEEE, 2025, pp. 345–350.
- [23] A. Hermans, L. Beyer, B. Leibe, In defense of the triplet loss for person re-identification, arXiv preprint arXiv:1703.07737 (2017).
- [24] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan, Supervised contrastive learning, Advances in neural information processing systems 33 (2020) 18661–18673.
- [25] elastic, What is elastic search? (2023).
URL <https://www.elastic.co/what-is/elasticsearch>
- [26] J. Luxemburk, T. Čejka, Fine-grained tls services classification with reject option, Computer Networks 220 (2023) 109467.
doi:<https://doi.org/10.1016/j.comnet.2022.109467>.
URL <https://www.sciencedirect.com/science/article/pii/S1389128622005011>
- [27] Radcom Ltd., Cellular network traffic dataset, Proprietary dataset. Radcom Ltd., Israel, not publicly available; used under data-sharing agreement with traffic anonymized at collection (2025).
- [28] Encrypted traffic classification: Supplementary results, <https://anonymous.4open.science/r/etc-supplementary-results-8426/>, anonymized repository for review (2026).